

Part I: Concepts & Theory

Q1. ACID Properties

The four ACID properties ensure that database transactions execute reliably.

1. **Atomicity:** A transaction is treated as a single indivisible unit. Either every operation succeeds or none of them take effect.

Example: During an online payment, if the amount is deducted from the customer's account but the merchant account cannot be credited, the deduction is undone.

2. **Consistency:** A transaction always moves the database from one valid state to another while preserving all constraints.

Example: A bank transfer should keep the total money in both accounts unchanged after completion.

3. **Isolation:** Concurrent transactions should not interfere with each other, making each transaction appear to execute independently.

Example: Two customers trying to buy the last product should not both successfully purchase it.

4. **Durability:** Once a transaction is committed, its changes remain permanent even if the system crashes immediately afterward.

Example: After an ATM confirms cash withdrawal, the account balance remains updated even if power fails.

Q2. Crash Before COMMIT

If the database crashes after the first **UPDATE** but before the **COMMIT**, the transaction is considered incomplete. During recovery, the DBMS performs a **ROLLBACK**, undoing all changes made by the transaction. Therefore, the money deducted from Account 101 is restored, ensuring that no partial transfer remains.

COMMIT permanently saves all modifications made during a transaction.

ROLLBACK cancels all changes made since the transaction began and restores the previous consistent state.

COMMIT is used when every statement executes successfully, while **ROLLBACK** is used whenever an error, crash, or unexpected condition prevents successful completion.

Q3. Concurrent Bank Access

The phenomenon observed is a **Non-Repeatable Read**. User A reads the same row twice but gets different values because another transaction modified and committed the data between the two reads.

This issue is controlled by the **Isolation** property of ACID. Appropriate transaction isolation levels such as **REPEATABLE READ** or **SERIALIZABLE** prevent this inconsistency.

Q4. SQL View

A View is a virtual table created from the result of an SQL query. It stores the query rather than the actual data.

Advantages:

- Improves security by exposing only selected columns or rows.
- Simplifies complex queries by hiding joins and calculations.

Data cannot always be updated through a view. Generally, a view is updatable only if it is based on a single table without aggregate functions, GROUP BY, DISTINCT, UNION, or complex joins, and each row in the view maps directly to one row in the underlying table.

Q5. Common Table Expression (CTE)

A Common Table Expression (CTE) is a temporary named result set defined using the WITH clause and is available only during execution of a single SQL statement.

Compared to ordinary subqueries, CTEs improve readability and allow the same intermediate result to be referenced multiple times.

A **non-recursive CTE** executes only once and returns a fixed result.

A **recursive CTE** repeatedly references itself until a stopping condition is met.

A common application of recursive CTEs is traversing hierarchical structures such as an employee-management hierarchy or folder directory tree.

Q6. Stored Procedure

A Stored Procedure is a collection of SQL statements stored inside the database that can be executed whenever required.

Advantages:

- Reduces network traffic because multiple SQL operations are executed with a single procedure call.
- Improves security by allowing users to execute procedures without direct access to database tables.

Part II: Practical SQL Application

Q7. Create View

```
CREATE VIEW SalesEmployees AS
SELECT EmpID, Name, Salary
FROM Employees
WHERE Department = 'Sales';
```

Q8. Transaction

```
BEGIN;  
  
UPDATE Employees  
SET Salary = Salary + 5000  
WHERE Name = 'Eve';  
  
COMMIT;
```

Q9. CTE Query

```
WITH AvgSalary AS (  
    SELECT AVG(Salary) AS AverageSalary  
    FROM Employees  
)  
  
SELECT Name, Salary  
FROM Employees, AvgSalary  
WHERE Salary > AverageSalary;
```

Part III: Long Answer

Q10(A)

These four operations should be executed as a single transaction because they are logically dependent on one another. If any one operation fails while the others succeed, the database becomes inconsistent. For example, recording payment without creating an order or reducing inventory would lead to incorrect business records. Treating all four operations as one transaction guarantees that either all of them succeed together or none of them take effect.

Q10(B)

Atomicity: Ensures that inventory deduction, payment recording, order creation, and order history update either all succeed or all fail together.

Consistency: Guarantees that inventory counts, payment records, and customer history satisfy all database constraints before and after the transaction.

Isolation: Prevents simultaneous customer transactions from interfering with one another. This avoids problems such as multiple customers purchasing the same remaining item.

Durability: Once an order is confirmed, all related information remains permanently stored even if the server crashes immediately afterward.

Q10(C)

Without Atomicity, suppose inventory is reduced and payment is recorded successfully, but the order entry fails because of a server crash. The customer would be charged and

inventory would decrease even though no order exists in the system. Recovering from such an inconsistency would require manual intervention and could result in customer complaints or financial loss.

Q10(D)

Isolation ensures that thousands of transactions executing simultaneously do not interfere with one another. Each customer's order behaves as though it is executing independently.

One serious concurrency problem is a **Dirty Read**. Suppose one transaction temporarily reduces inventory but has not yet committed. Another transaction reads this temporary value and incorrectly concludes that an item is out of stock. If the first transaction later rolls back, the second transaction has already acted on incorrect information, leading to inaccurate inventory management and poor customer experience.